

ClassRoomPB

Sistema Acadêmico — ES2

Relatório de Processo da Release 2

Sprint 3 + Sprint 4

Tecnologia: Java | Arquitetura: MVC | Persistência: JSON

Equipe: Eric Natan · José Eudson · Rui Fernando

2026

1. Visão Geral da Release 2

A Release 2 do sistema ClassRoomPB compreende duas sprints de desenvolvimento, realizadas entre 02/06/2026 e 29/06/2026. O objetivo desta release é implementar o fluxo principal do aluno no sistema: desde a consulta de turmas disponíveis e solicitação de matrícula, passando pela gestão de lista de espera, até o acompanhamento de frequência. Além disso, a release inclui a definição e aplicação de um guia de estilo visual ao projeto.

Sprint 3	Sprint 4	RFs Totais	Período
RF15–RF23 (9 RFs)	RF24–RF30 (7 RFs)	16 Requisitos Funcionais	02/06 – 29/06/2026

2. Tecnologias Utilizadas

Tecnologia / Ferramenta	Detalhe
Linguagem	Java
Arquitetura	MVC (Model, View, Controller)
Build / Deps	Maven
Testes	JUnit 5 (junit-jupiter)
Cobertura	JaCoCo
Persistência	JSON (arquivos .json por entidade)
Mocks	Mockito 4.11.0
Guia de Estilo	Definição e aplicação de convenções visuais e de código

3. Processo de Desenvolvimento e Organização da Equipe

A equipe manteve a mesma estratégia adotada na Release 1: distribuição de tarefas baseada em estimativa de horas por task, com o objetivo de equilibrar a carga de trabalho entre os membros. Antes de cada sprint, a equipe se reuniu para decompor os requisitos funcionais em tasks menores e atribuir estimativas de esforço.

A rastreabilidade entre planejamento e execução continuou sendo mantida pelo Azure DevOps, com cada requisito funcional cadastrado como item de trabalho e o link para o commit correspondente registrado na descrição do RF no repositório do GitHub.

3.1 Sprint 3 (02/06/2026 – 11/06/2026)

Gerente da Sprint: Rui Fernando | QA: Eric Natan | Dev: José Eudson

Membro	RF / Tasks	Entregas
Eric Natan	RF15, RF16, RF17 / Tasks 2.1.1–2.1.6 + Atividade de QA	Consulta de turmas, solicitação de matrícula, verificação de vagas, testes e revisão de qualidade
José Eudson	RF21, RF22, RF23 / Tasks 2.1.13–2.1.18	Lista de espera, cancelamento de matrícula, manutenção da lista de espera, testes
Rui Fernando	RF18, RF19, RF20 / Tasks 2.1.7–2.1.12	Verificação de pré-requisitos, choque de horário do aluno, confirmação automática de matrícula, testes

Atribuições de Eric Natan (Sprint 3)

RF15 — Consulta de Disciplinas/Turmas Disponíveis:

- Task 2.1.1: Implementação da consulta de disciplinas/turmas disponíveis, com interface de linha de comando para listagem filtrada por período e disciplina.
- Task 2.1.2: Testes unitários para consulta de turmas, cobrindo cenários com e sem turmas disponíveis.

RF16 — Solicitação de Matrícula:

- Task 2.1.3: Implementação da solicitação de matrícula pelo aluno, integrando as validações de negócio ao fluxo de confirmação.
- Task 2.1.4: Testes unitários para solicitação de matrícula.

RF17 — Verificação de Vagas Disponíveis (RN03):

- Task 2.1.5: Implementação da verificação automática de vagas disponíveis antes de confirmar a matrícula.
- Task 2.1.6: Testes unitários para verificação de vagas, incluindo turma cheia e turma com vaga.

Atividade de QA (Sprint 3):

- Revisão e validação dos entregáveis da sprint, garantindo cobertura de testes adequada e consistência entre os módulos desenvolvidos pelos demais membros.

Atribuições de José Eudson (Sprint 3)

RF21 — Entrada do Aluno na Lista de Espera:

- Task 2.1.13: Implementação da adição automática do aluno à lista de espera quando não há vagas disponíveis na turma.
- Task 2.1.14: Testes unitários para adição à lista de espera.

RF22 — Cancelamento de Matrícula:

- Task 2.1.15: Implementação do cancelamento de matrícula dentro do período permitido, com liberação automática de vaga.
- Task 2.1.16: Testes unitários para cancelamento de matrícula.

RF23 — Manutenção da Lista de Espera por Turma:

- Task 2.1.17: Implementação da manutenção da lista de espera por turma, incluindo persistência e consulta.
- Task 2.1.18: Testes unitários para manutenção da lista de espera.

Atribuições de Rui Fernando (Sprint 3)

RF18 — Verificação de Pré-requisitos (RN04, RN05):

- Task 2.1.7: Implementação da verificação automática de pré-requisitos da disciplina, bloqueando matrícula quando não cumpridos.
- Task 2.1.8: Testes unitários para verificação de pré-requisitos.

RF19 — Impedimento de Choque de Horário do Aluno (RN02):

- Task 2.1.9: Implementação da verificação de choque de horário entre turmas do mesmo aluno no mesmo período letivo.
- Task 2.1.10: Testes unitários para validação de choque de horário.

RF20 — Confirmação Automática de Matrícula:

- Task 2.1.11: Implementação da confirmação automática de matrícula quando todos os critérios são atendidos.
- Task 2.1.12: Testes unitários para confirmação automática.

Tasks de Suporte — Sprint 3:

- Task S2.1: Implementação da camada de persistência para Matrículas e Lista de Espera.
- Task S2.2: Implementação da interface de linha de comando para consulta e solicitação de matrícula.
- Task S2.3: Tratamento de erros gerais para matrícula e lista de espera.
- Task DOC.S3: Elaboração do relatório de gerenciamento do processo da Sprint 3.

3.2 Sprint 4 (12/06/2026 – 29/06/2026)

Gerente da Sprint: Eric Natan | QA: José Eudson | Dev: Rui Fernando

Membro	RF / Tasks	Entregas
Eric Natan	RF24, RF25 / Tasks 2.2.1–2.2.4 + Guia de Estilo + Documentações	Chamada automática da lista de espera, ordem de solicitação, definição do guia de estilo, elaboração dos relatórios
José Eudson	RF26, RF27 / Tasks 2.2.5–2.2.9 + Atividade de QA	Visualização da lista de espera pelo coordenador, registro de presença/falta pelo professor, testes e revisão de qualidade
Rui Fernando	RF28, RF29, RF30 / Tasks 2.2.10–2.2.15	Cálculo de frequência, consulta de frequência pelo aluno, alerta de baixa frequência, testes

Atribuições de Eric Natan (Sprint 4)

RF24 — Chamada Automática da Lista de Espera:

- Task 2.2.1: Implementação da chamada automática do próximo aluno da lista de espera quando uma vaga é liberada por cancelamento de matrícula.
- Task 2.2.2: Testes unitários para chamada automática, cobrindo cenários com e sem candidatos na lista.

RF25 — Ordem de Solicitação na Lista de Espera:

- Task 2.2.3: Implementação da regra que garante o respeito à ordem cronológica de solicitação na lista de espera.
- Task 2.2.4: Testes unitários para ordem da lista de espera.

Guia de Estilo do Projeto:

- Pesquisa e definição de convenções visuais e de código para o projeto ClassRoomPB, incluindo padrões de nomenclatura, formatação de saída no console e organização de pacotes.
- Aplicação e revisão do guia de estilo nos módulos existentes para garantir consistência visual e de código em todo o projeto.

Documentações:

- Elaboração do relatório de processo da Sprint 4 e da Release 2.

Atribuições de José Eudson (Sprint 4)

RF26 — Visualização da Lista de Espera pelo Coordenador:

- Task 2.2.5: Implementação da funcionalidade que permite ao Coordenador visualizar a lista de espera de cada turma, exibindo a ordem e os dados dos candidatos.
- Task 2.2.6: Testes unitários para visualização da lista de espera.

RF27 — Registro de Presença/Falta pelo Professor:

- Task 2.2.7: Modelagem de classes para Frequência, incluindo entidade, repositório e serviço.
- Task 2.2.8: Implementação do registro de presença/falta dos alunos por aula pelo Professor.
- Task 2.2.9: Testes unitários para registro de frequência.

Atividade de QA (Sprint 4):

- Revisão e validação dos entregáveis da sprint, com execução dos testes de integração da Release 2.

Atribuições de Rui Fernando (Sprint 4)

RF28 — Cálculo Automático do Percentual de Frequência:

- Task 2.2.10: Implementação do cálculo automático do percentual de frequência por aluno e disciplina, com base nos registros de presença/falta.
- Task 2.2.11: Testes unitários para cálculo de frequência.

RF29 — Consulta de Frequência pelo Aluno:

- Task 2.2.12: Implementação da consulta de frequência por disciplina no menu do Aluno.
- Task 2.2.13: Testes unitários para consulta de frequência.

RF30 — Alerta de Baixa Frequência (RN08):

- Task 2.2.14: Implementação do alerta automático quando o percentual de frequência do aluno está abaixo do mínimo exigido pela regra de negócio RN08.
- Task 2.2.15: Testes unitários para alerta de baixa frequência.

Tasks de Suporte — Sprint 4:

- Task S2.4: Implementação da camada de persistência para Frequência.
- Task S2.5: Implementação da interface de linha de comando para registro e consulta de frequência.
- Task S2.6: Tratamento de erros gerais para frequência.
- Task S2.7: Revisão e execução de testes de integração para Release 2.
- Task DOC.S4: Elaboração do relatório de gerenciamento do processo da Sprint 4.
- Task DOC.R2: Elaboração do relatório da Release 2 (arquitetura, funcionalidades, testes e cobertura).

4. Burndown das Sprints

4.1 Sprint 3 — Burndown

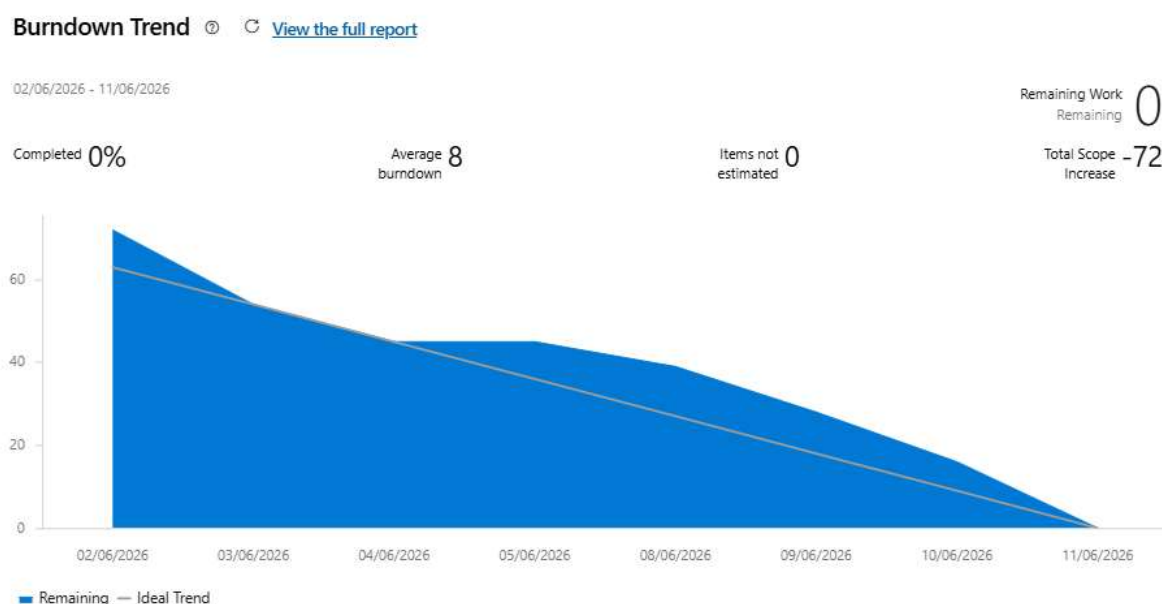


Figura 1: Gráfico de Burndown da Sprint 3

4.2 Sprint 4 — Burndown

Burndown Trend [View the full report](#)

12/06/2026 - 29/06/2026

Remaining Work
Remaining 0

Completed 0%

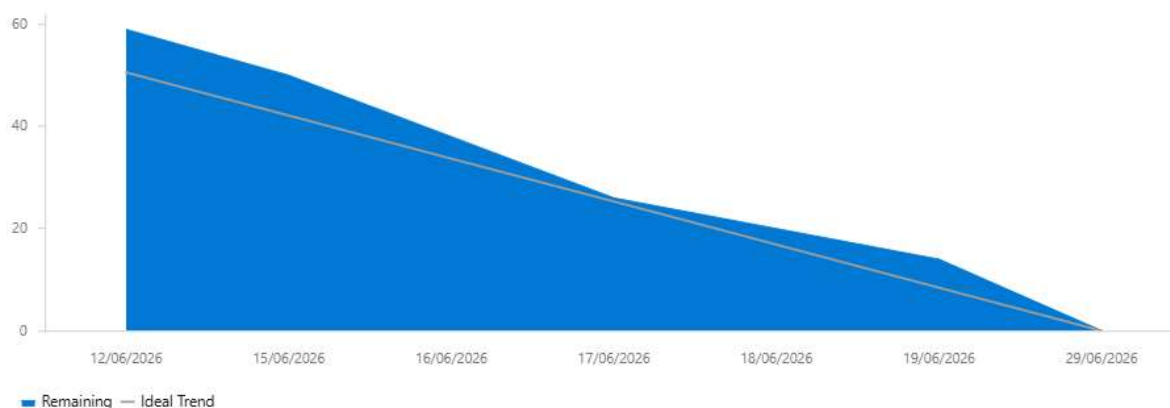
Average
burndown 3Items not
estimated 0Total Scope
Increase -59

Figura 2: Gráfico de Burndown da Sprint 4

5. Registro das Funcionalidades

5.1 Sprint 3 — RF15 a RF23

RF	Descrição	Responsável
RF15	Consulta de disciplinas/turmas disponíveis pelo aluno	Eric Natan
RF16	Solicitação de matrícula em uma turma pelo aluno	Eric Natan
RF17	Verificação de vagas disponíveis (RN03)	Eric Natan
RF18	Verificação de pré-requisitos da disciplina (RN04, RN05)	Rui Fernando
RF19	Impedimento de choque de horário entre turmas do mesmo aluno (RN02)	Rui Fernando
RF20	Confirmação automática de matrícula quando todos os critérios são atendidos	Rui Fernando
RF21	Entrada automática do aluno em lista de espera quando não há vagas	José Eudson
RF22	Cancelamento de matrícula dentro do período permitido	José Eudson
RF23	Manutenção da lista de espera por turma	José Eudson

5.2 Sprint 4 — RF24 a RF30

RF	Descrição	Responsável
RF24	Chamada automática do próximo aluno da lista de espera ao liberar vaga	Eric Natan
RF25	Lista de espera respeita a ordem de solicitação	Eric Natan
RF26	Visualização da lista de espera de cada turma pelo Coordenador	José Eudson
RF27	Registro de presença/falta dos alunos por aula pelo Professor	José Eudson
RF28	Cálculo automático do percentual de frequência	Rui Fernando
RF29	Consulta de frequência por disciplina pelo aluno	Rui Fernando
RF30	Alerta ao aluno quando frequência está abaixo do mínimo exigido (RN08)	Rui Fernando

6. Detalhamento das Funcionalidades — Sprint 3

6.1 Consulta, Solicitação de Matrícula e Verificação de Vagas (RF15, RF16 e RF17)

O sistema disponibiliza ao Aluno um menu de matrícula que permite consultar as disciplinas e turmas disponíveis no período letivo ativo (RF15). A listagem exibe código, horário, sala, professor responsável e número de vagas remanescentes, permitindo que o aluno tome uma decisão informada antes de solicitar a matrícula.

A solicitação de matrícula (RF16) é iniciada pelo aluno informando o código da turma desejada. O sistema executa em sequência todas as validações de negócio obrigatórias antes de efetivar ou recusar a matrícula. A verificação de vagas (RF17) é a primeira regra avaliada: se não houver vaga disponível, o aluno é automaticamente encaminhado para a lista de espera, conforme RF21.

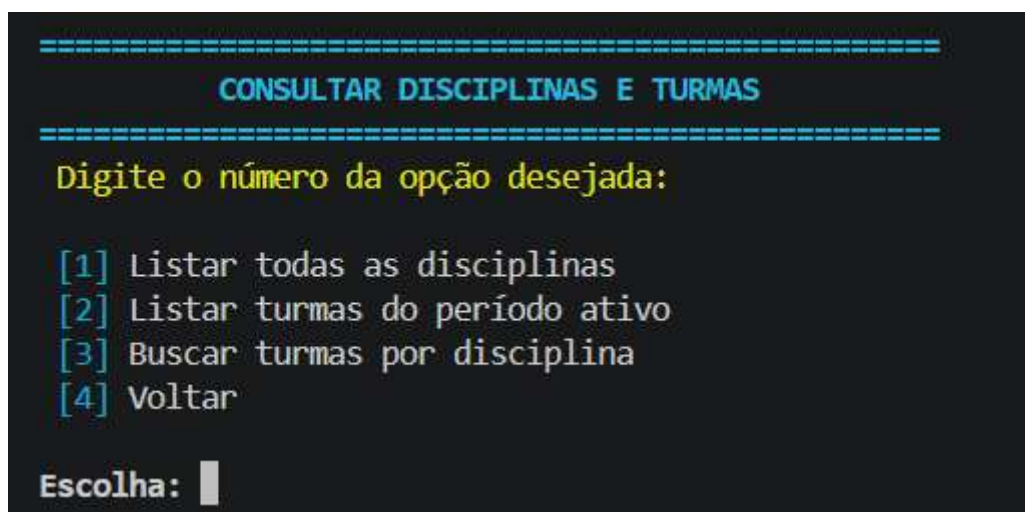


Figura 3: Menu de consulta pelo Aluno

6.2 Verificação de Pré-requisitos e Choque de Horário (RF18 e RF19)

Após a verificação de vagas, o sistema valida se o aluno já cumpriu todos os pré-requisitos da disciplina (RF18, regras RN04 e RN05). Essa verificação consulta o histórico acadêmico do aluno e compara com a lista de pré-requisitos da disciplina. Caso algum pré-requisito não tenha sido cumprido, a matrícula é recusada com mensagem descritiva indicando qual disciplina ainda precisa ser cursada.

A verificação de choque de horário (RF19, regra RN02) garante que o aluno não seja matriculado em duas turmas com horários conflitantes no mesmo período letivo. O sistema verifica os horários de todas as turmas em que o aluno já está matriculado e rejeita a solicitação caso haja sobreposição, informando a turma conflitante.

```
=====
TURMAS DISPONÍVEIS ? C2
=====
+-----+-----+-----+-----+-----+-----+
| Código | Disciplina | Horário      | Sala      | Vagas | Professor |
+-----+-----+-----+-----+-----+-----+
| T01    | C2         | Ter/Sex 9h-11h | Bloco C - 107 | 15    | P0002     |
+-----+-----+-----+-----+-----+-----+

Código da turma desejada: T01

[!] Erro: O aluno nao foi aprovado no pre-requisito C1
Pressione ENTER para continuar...
█
```

Figura 4: Mensagem de erro ao tentar se matricular sem cumprir pré-requisito

6.3 Confirmação Automática de Matrícula (RF20)

Quando todas as validações são satisfeitas — vaga disponível, pré-requisitos cumpridos e sem choque de horário — o sistema confirma automaticamente a matrícula do aluno na turma (RF20). O registro é persistido no arquivo matriculas.json, a contagem de vagas é atualizada na turma e uma mensagem de confirmação é exibida ao aluno com os dados da matrícula efetivada.

```
=====
TURMAS DISPONÍVEIS ? C1
=====
+-----+-----+-----+-----+-----+-----+
| Código | Disciplina | Horário      | Sala      | Vagas | Professor |
+-----+-----+-----+-----+-----+-----+
| T01    | C1         | Seg/Qua 11h-13h | Bloco C - 106 | 25    | P0001     |
+-----+-----+-----+-----+-----+-----+

Código da turma desejada: T01

[?] Matrícula confirmada com sucesso! Turma: T01 | Disciplina: C1 | Período: 2026.1.
Pressione ENTER para continuar...
█
```

Figura 5: Confirmação automática de matrícula

6.4 Lista de Espera e Cancelamento de Matrícula (RF21, RF22 e RF23)

Quando não há vagas na turma desejada, o aluno é automaticamente incluído na lista de espera daquela turma (RF21), com registro do momento da solicitação para garantir a prioridade cronológica. O Coordenador pode visualizar e acompanhar a lista de espera de qualquer turma (RF23), que é mantida em arquivo dedicado `listasespera.json`.

O cancelamento de matrícula (RF22) é permitido ao aluno dentro do período definido pelo calendário acadêmico. Ao cancelar, a vaga é liberada e o sistema aciona automaticamente o mecanismo de chamada da lista de espera (integração com RF24).

7. Detalhamento das Funcionalidades — Sprint 4

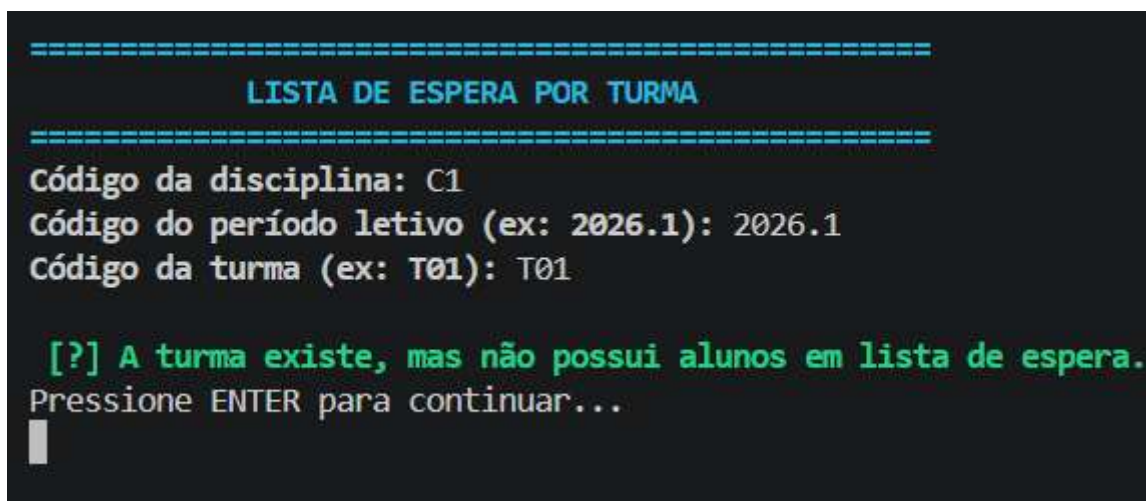
7.1 Chamada Automática da Lista de Espera (RF24 e RF25)

Quando uma vaga é liberada por cancelamento de matrícula, o sistema chama automaticamente o próximo aluno da lista de espera daquela turma (RF24). A chamada respeita estritamente a ordem cronológica de solicitação (RF25): o primeiro a entrar na lista é o primeiro a ser convocado. O aluno convocado tem sua matrícula confirmada automaticamente, e a lista de espera é atualizada.

Caso nenhum aluno esteja na lista de espera, a vaga permanece disponível normalmente para novas solicitações. O processo é implementado no `MatriculaService` e integrado ao fluxo de cancelamento.

7.2 Visualização da Lista de Espera pelo Coordenador (RF26)

O Coordenador pode acessar, pelo menu de gerenciamento de turmas, a lista de espera de qualquer turma do período ativo (RF26). A listagem exibe a posição de cada candidato, sua matrícula, nome e o momento em que entrou na fila, permitindo acompanhamento e auditoria do processo.



```
=====
LISTA DE ESPERA POR TURMA
=====
Código da disciplina: C1
Código do período letivo (ex: 2026.1): 2026.1
Código da turma (ex: T01): T01

[?] A turma existe, mas não possui alunos em lista de espera.
Pressione ENTER para continuar...
█
```

Figura 6: Visualização da lista de espera pelo Coordenador

7.3 Registro de Presença/Falta pelo Professor (RF27)

O Professor pode registrar a presença ou falta de cada aluno matriculado em suas turmas por aula (RF27). Para cada sessão, o sistema lista os alunos da turma e permite que o professor marque individualmente a situação de cada um. Os registros são persistidos no arquivo `frequencias.json` e servem de base para o cálculo automático de frequência.

A modelagem de classes para Frequência inclui a entidade Frequencia com campos de data da aula, código da turma, matrícula do aluno e status de presença/falta. O FrequenciaRepository gerencia a persistência e o FrequenciaService orquestra as operações de registro e consulta.

[Inserir aqui imagem do registro de presença pelo Professor]

Figura 7: Interface de registro de presença/falta pelo Professor

7.4 Cálculo de Frequência, Consulta e Alerta (RF28, RF29 e RF30)

O sistema calcula automaticamente o percentual de frequência de cada aluno por disciplina (RF28), dividindo o número de presenças pelo total de aulas registradas até o momento. O cálculo é atualizado a cada novo registro e fica disponível para consulta imediata.

O Aluno pode consultar sua frequência em cada disciplina pelo menu de acompanhamento (RF29), visualizando o percentual atual, o número de presenças, de faltas e o total de aulas. Quando o percentual está abaixo do mínimo exigido pela regra RN08, o sistema exibe um alerta destacado ao aluno (RF30), indicando o risco de reprovação por falta e a quantidade de aulas que pode ainda faltar sem perder o mínimo.

[Inserir aqui imagem da consulta de frequência e alerta de baixa frequência]

Figura 8: Consulta de frequência pelo Aluno e alerta de baixa frequência

7.5 Guia de Estilo do Projeto

Na Sprint 4, Eric Natan conduziu a pesquisa e definição do guia de estilo do projeto ClassRoomPB. O guia estabelece convenções para: formatação das mensagens exibidas no console (uso de separadores, prefixos de status [?], [!], emojis e cores ANSI); nomenclatura de classes, métodos e atributos em Java; organização e responsabilidades dos pacotes da arquitetura MVC; e padrões para nomes de arquivos JSON de persistência.

Após a definição, o guia foi aplicado retroativamente aos módulos da Release 1 e nos novos módulos da Release 2, garantindo uma experiência de uso e de manutenção de código consistente em todo o projeto. As ferramentas de qualidade integradas ao Maven (Checkstyle, PMD, CPD, SpotBugs, JaCoCo e formatter-maven-plugin) automatizam a verificação das convenções de código a cada build, enquanto as convenções de produto a seguir foram definidas manualmente pela equipe.

Convenções de Nomenclatura (Checkstyle)

A nomenclatura de classes, métodos e atributos em Java é verificada automaticamente pelo Checkstyle a cada build (mvn validate):

Elemento	Convenção	Exemplo
Classes e interfaces	PascalCase	UsuarioService
Métodos	camelCase	buscarPorEmail()
Variáveis e parâmetros	camelCase	nomeAluno
Constantes	UPPER_SNAKE_CASE	MAX_VAGAS
Pacotes	minúsculas	com.classroompb.service

Regras adicionais: proibido o uso de import * no código de produção, chaves obrigatórias em if/else/for/while, proibida a comparação de Strings com ==, e complexidade ciclomática máxima de 25 por método. Exceção para testes: arquivos *Test.java permitem import static.*

de Assertions e nomes de método com underscore, seguindo o padrão BDD `deveFazerX_quandoY`.

Organização de Pacotes (Arquitetura MVC)

O guia define a responsabilidade de cada pacote da arquitetura MVC do projeto, evitando que lógica de negócio fique fora da camada de serviço:

- **model** — entidades de domínio (ex.: `Matricula`, `ListaEspera`, `Frequencia`), sem lógica de negócio.
- **repository** — persistência e acesso aos arquivos JSON (leitura, escrita e consulta).
- **service** — regras de negócio (RNs), orquestração entre repositórios e validações.
- **view / ui (CLI)** — interface de linha de comando, exibição de menus e mensagens ao usuário.
- **util** — classes auxiliares reutilizáveis (formatação, validação genérica, etc.).
- **exception** — exceções customizadas do domínio.

Regra geral: a camada view nunca acessa repository diretamente — toda chamada passa por service.

Convenções de Formatação de Saída no Console

Para manter a experiência de uso consistente entre os menus de Aluno, Professor e Coordenador, o guia define:

- **Separadores** — linhas de `"=` ou `"-` para delimitar blocos de menu e seções de saída.
- **Prefixos de status** — `[?]` para pergunta ou entrada esperada do usuário; `[!]` para alerta ou erro de validação (ex.: matrícula recusada, frequência baixa).
- **Emojis** — uso pontual em mensagens de destaque (ex.: alerta de baixa frequência na RF30), sem exagero, para não comprometer a legibilidade em terminais sem suporte a unicode.
- **Cores ANSI** — verde para sucesso, vermelho para erro/alerta e amarelo para avisos, aplicadas via códigos de escape ANSI padronizados em uma classe utilitária comum, nunca de forma fixa em cada classe de interface.

Padrões de Nomenclatura para Arquivos JSON de Persistência

O guia também padroniza os nomes dos arquivos JSON usados na camada de persistência: nome no plural, em minúsculas e em português (ex.: `matriculas.json`, `listasespera.json`, `frequencias.json`); um arquivo dedicado por entidade ou agregado, sem misturar entidades distintas no mesmo arquivo; e nomes sem espaços ou acentos, para evitar problemas de compatibilidade entre sistemas operacionais.

8. Testes Unitários

Todos os requisitos implementados em ambas as sprints possuem cobertura de testes unitários, escritos com o framework JUnit 5 (junit-jupiter) e executados via Maven (mvn test). Os testes continuaram organizados em classes aninhadas (@Nested) para separar claramente os cenários de cada funcionalidade.

8.1 Testes da Sprint 3

Testes de Matrícula (RF15, RF16, RF17, RF20)

- Consulta de turmas disponíveis com e sem turmas no período ativo.
- Solicitação de matrícula com sucesso quando todos os critérios são atendidos.
- Rejeição de matrícula por turma sem vagas e encaminhamento para lista de espera.
- Confirmação automática após aprovação em todas as validações.

Testes de Pré-requisitos e Choque de Horário (RF18 e RF19)

- Matrícula rejeitada quando pré-requisito não cumprido, com identificação da disciplina bloqueante.
- Matrícula rejeitada com choque de horário, informando a turma conflitante.
- Matrícula permitida quando pré-requisitos cumpridos e sem conflito de horário.

Testes de Lista de Espera e Cancelamento (RF21, RF22 e RF23)

- Aluno adicionado à lista de espera quando turma está cheia.
- Cancelamento de matrícula com sucesso dentro do período permitido.
- Rejeição de cancelamento fora do período permitido.
- Lista de espera mantida por turma com persistência correta.

8.2 Testes da Sprint 4

Testes de Chamada da Lista de Espera (RF24 e RF25)

- Chamada automática do próximo candidato ao liberar vaga por cancelamento.
- Respeito à ordem cronológica na chamada dos candidatos da lista de espera.
- Nenhuma ação realizada quando a lista de espera está vazia.

Testes de Visualização da Lista de Espera (RF26)

- Listagem correta dos candidatos em espera por turma.
- Acesso permitido apenas ao Coordenador.

Testes de Registro de Presença/Falta (RF27)

- Registro de presença e falta com persistência correta no repositório.
- Validação de acesso: apenas Professores podem registrar frequência.
- Rejeição de registro para aluno não matriculado na turma.

Testes de Cálculo de Frequência e Alerta (RF28, RF29 e RF30)

- Cálculo correto do percentual de frequência em diferentes cenários.
- Consulta de frequência exibindo percentual atualizado ao aluno.
- Alerta disparado quando percentual abaixo do mínimo definido pela RN08.
- Alerta não disparado quando frequência está dentro do limite aceitável.

8.3 Cobertura de Testes — JaCoCo (Sprint 3)

classroompb

classroompb

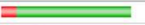
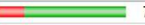
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.classroompb.service		87%		77%	118	387	90	711	7	92	0	10
com.classroompb.repository		92%		83%	13	122	23	276	4	94	1	7
com.classroompb.model		91%		83%	9	110	14	185	8	104	0	13
com.classroompb.util		91%		85%	5	18	3	34	2	8	0	2
com.classroompb.exception		100%		100%	0	6	0	11	0	5	0	2
Total	533 of 5.021	89%	144 of 668	78%	145	643	130	1.217	21	303	1	34

Figura 9: Relatório de cobertura de testes — JaCoCo (Sprint 3)

8.4 Cobertura de Testes — JaCoCo (Sprint 4)

classroompb

classroompb

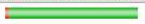
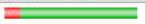
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.classroompb.service		96%		87%	75	416	31	798	3	111	0	11
com.classroompb.model		89%		81%	9	133	16	261	7	125	0	15
com.classroompb.repository		97%		85%	12	153	4	265	0	111	0	8
com.classroompb.util		91%		83%	6	20	3	34	2	8	0	2
com.classroompb.exception		100%		100%	0	6	0	11	0	5	0	2
Total	251 of 5.923	95%	93 of 724	87%	102	728	54	1.369	12	360	0	38

Figura 10: Relatório de cobertura de testes — JaCoCo (Sprint 4)

9. Persistência de Dados

O sistema ClassRoomPB manteve o JSON como mecanismo de persistência. A Release 2 adicionou três novos arquivos de persistência ao projeto, cobrindo as entidades Matrícula, Lista de Espera e Frequência.

Arquivo	Entidade	Responsável	Sprint
usuarios.json	Usuario (todos os perfis)	Eric Natan	Sprint 1
cursos.json	Curso	José Eudson	Sprint 1
disciplinas.json	Disciplina	José Eudson / Rui Fernando	Sprint 1
periodos_letivos.json	PeriodoLetivo	Rui Fernando	Sprint 1
turmas.json	Turma	Eric Natan	Sprint 2
matriculas.json	Matricula	Eric Natan	Sprint 3
listasespera.json	ListaEspera	José Eudson	Sprint 3
frequencias.json	Frequencia	Rui Fernando	Sprint 4

9.1 Exemplos de Conteúdo dos Arquivos JSON

```
{ } matriculas_turma.json > ...
1  [ {
2    "matriculaAluno" : "A0001",
3    "codigoDisciplina" : "EDA",
4    "codigoPeriodo" : "2026.1",
5    "codigoTurma" : "T01",
6    "status" : "CANCELADA",
7    "dataSolicitacao" : "2026-06-03T18:36:12.0307268"
8  }, {
9    "matriculaAluno" : "A0001",
10   "codigoDisciplina" : "GP",
11   "codigoPeriodo" : "2026.1",
12   "codigoTurma" : "T01",
13   "status" : "CONFIRMADA",
14   "dataSolicitacao" : "2026-06-03T18:36:46.3637582"
15 }, {
16   "matriculaAluno" : "A0001",
17   "codigoDisciplina" : "EDA",
18   "codigoPeriodo" : "2026.1",
19   "codigoTurma" : "T01",
20   "status" : "PENDENTE",
21   "dataSolicitacao" : "2026-06-04T10:41:43.3438611"
22 }, {
23   "matriculaAluno" : "A0001",
24   "codigoDisciplina" : "AL",
25   "codigoPeriodo" : "2026.1",
26   "codigoTurma" : "T01",
27   "status" : "CONFIRMADA",
28   "dataSolicitacao" : "2026-06-11T17:00:27.1885395"
29 }, {
30   "matriculaAluno" : "A0001",
31   "codigoDisciplina" : "C1",
32   "codigoPeriodo" : "2026.1",
33   "codigoTurma" : "T01",
34   "status" : "CONFIRMADA",
35   "dataSolicitacao" : "2026-06-29T10:20:53.8205527"
36 } ]
```

Figura 13: Conteúdo do arquivo matriculas_turma.json

```
=====
LISTA DE ESPERA POR TURMA
=====
Código da disciplina: C1
Código do período letivo (ex: 2026.1): 2026.1
Código da turma (ex: T01): T02
```

#	Aluno	Disciplina	Período	Turma	Status	Data
1	A0001	C1	2026.1	T02	LISTA_ESPERA	03/07/2026 09:20

Total de alunos em lista de espera: 1

```
{
  "matriculaAluno" : "A0001",
  "codigoDisciplina" : "C1",
  "codigoPeriodo" : "2026.1",
  "codigoTurma" : "T02",
  "status" : "LISTA_ESPERA",
  "dataSolicitacao" : "2026-07-03T09:20:16.053221"
}
```

Figura 14: lista de espera

```
{ } frequencias.json > ...
1 [ {
2   "matriculaAluno" : "A0001",
3   "codigoDisciplina" : "GP",
4   "codigoPeriodo" : "2026.1",
5   "codigoTurma" : "T01",
6   "dataAula" : "2026-07-03",
7   "status" : "PRESENTE",
8   "matriculaProfessor" : "P0001",
9   "dataRegistro" : "2026-07-03T09:14:01.9467267"
10 }
```

Figura 15: Conteúdo do arquivo frequencias.json